

离散数学

刘晓芳

liuxiaofang@nankai.edu.cn



机器人与信息自动化研究所

Institute of Robotics & Automatic Information System



南開大學

Nankai University

第3章

基础：算法和整数

主要内容

- 第3章 基础：算法和整数
 - 3.1 算法
 - 3.2 函数的增长率
 - 3.3 算法的复杂性
 - 3.4 整数与整除
 - 3.5 素数和最大公因子
 - 3.6 整数和算法
 - 3.7 数论的应用

3.1 算法 (Algorithm)

- 定义：算法是一个有穷的指令序列。每条指令必须有清楚的含义，并且在有穷长的时间用有穷的动作能完成。一个算法无论接受任何输入都必须在有穷步内停止。
- 例1：设计一个算法在有穷长的整数序列中找最大的那个整数。用类Pascal语言来描述。
- PROCEDURE max($a_1, a_2, \dots, a_n$: integers)
- max := a_1 ;
- FOR i := 1 TO n DO
- IF max < a_i THEN max := a_i ;
- {max is the largest element}

3.1 算法

- *算法一般具有以下性质：
 - (1) 输入：一个算法具有取自某一个指定集合的输入数据；
 - (2) 输出：一个算法产生取自某一个集合的输出数据，这些输出数据是问题的解；
 - (3) 确定性：算法的每一步都必须精确地定义；
 - (4) 正确性：算法应对每一组输入数据产生正确的输出数据
 - (5) 有穷性：输入数据后，算法应在有穷步后产生所期望的输出；
 - (6) 有效性：算法应能确切地在有穷时间内执行完每一步；
 - (7) 一般性：算法应该能对某种形式的所有问题求解，而不是只对某一组特殊的数据求解。

*例如：求有穷序列中的最大元问题。

3.1 算法

- 例2：线性查找算法。
- 问题：在某个有穷长的序列中，找其中是否有某个元素 x .
- PROCEDURE linear search(x : integer; $a_1, a_2, \dots, a_n$: distinct integers)
- $i := 1$;
- WHILE ($i \leq n$ and $x \neq a_i$) DO
- $i := i + 1$;
- IF $i \leq n$ THEN location := i
- ELSE location := 0;
- {location is the subscript of the term that equals x , or is 0 if x is not found}

3.1 算法

- 例3：二分查找：假设有穷序列已经从小到大排好序，找其中是否有某个元素 x 。
- 例如：在以下有序序列中找元素 $x=19$ 。
- 1, 2, 3, 5, 6, 7, 8, 10, 12, 13, 15, 16, 18, 19, 20, 22
- 将该序列分成两半(该序列原有16项,分成两半,各有8项)
- 1, 2, 3, 5, 6, 7, 8, 10; 12, 13, 15, 16, 18, 19, 20, 22;
- $x=19$ 与前一半的最后一项比较，如果 x 小于或等于该项，则在前一半中查找；如果 x 大于该项，则在后一半中查找。现在 $x=19>10$,故在后一半中查找。
- 再将后一半分成两半（各含4项元素）：
- 12, 13, 15, 16; 18, 19, 20, 22;

3.1 算法

- $x=19$ 与前半项的最后一项比较，如果 x 小于或等于该项，则在前一半中查找；如果 x 大于该项，则在后一半中查找。现在 $x=19>16$ ，故在后一半中查找。
- 再将后一半分成两半（各含2项元素）：
 $18, 19; \quad 20, 22;$
- 再用以上同样的比较方法，现在 $x=19\leq 19$ ，故在前一半中查找。再将前一半分成两半（各含1项元素）。
 $18; \quad 19;$
- 再用以上同样的比较方法，现在 $x=19>18$ ，故在后一半中查找。这时，后一半只剩一个元素19了， $x=19$ 与该项相同，因此查到该元素。

3.1 算法

- PROCEDURE binary search (x : integer; $a_1, a_2, \dots, a_n$: increasing integers)
- $i := 1$; { i is left endpoint of search interval}
- $j := n$; { j is right endpoint of search interval}
- WHILE $i < j$ DO
- BEGIN
- $m := \lfloor (i + j)/2 \rfloor$;
- IF $x > a_m$ THEN $i := m + 1$
- ELSE $j := m$;
- END;
- IF $x = a_i$ THEN location := i
- ELSE location := 0;
- {location is the subscript of the term that equals x , or 0 if x is not found}.

3.1 算法

- 贪心算法
- *最优化问题：有些问题是寻找一个问题的最优解，即该解使得问题的某个参数最大或最小。
- *贪心算法：在求解最优化问题时，每一步都取最合意的选择，这样的算法称为贪心算法。

3.1 算法

- 例6：假设现在要找 n 分钱的零钱，共有4种硬币，25分，10分，5分和1分，问怎样找钱，才能使所用的硬币数最少？
- 例如：找67分钱。先用最大的硬币：25分， $67 - 25 = 42$ 分，再用最大的硬币， $42 - 25 = 17$ 分；这时，用第二大的硬币， $17 - 10 = 7$ 分；再用第三大的硬币， $7 - 5 = 2$ 分；最后用两次最小的硬币， $2 - 1 = 1$ ，和 $1 - 1 = 0$ 分。这样找的硬币数恰好是最小的。

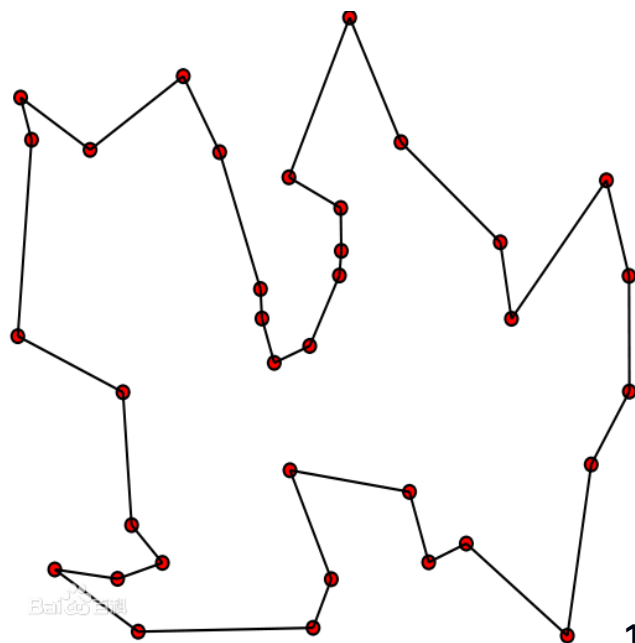
3.1 算法

- 算法6：（贪心找零钱算法）
- PROCEDURE change($c_1, c_2, \dots, c_r$: values of denominations of coins, where $c_1 > c_2 > \dots > c_r$; n : a positive integer);
- FOR $i := 1$ TO r DO
- WHILE $n \geq c_i$ DO
- BEGIN
- add a coin with value c_i to the change;
- $n := n - c_i$;
- END;
- 注意：贪心算法并不总能够求出最优解，贪心算法是否可用，必须证明其正确性才可用。

3.1 算法

- 例子：旅行商问题
- 给定一系列城市 and 每对城市之间的距离，求解访问每一座城市一次并回到起始城市的最短回路。它是组合优化中的一个NP难问题，在运筹学和理论计算机科学中非常重要。

- 贪心算法
- 遗传算法
- 粒子群优化
- 蚁群算法



3.2 函数的增长率

- 大O记号(Big-O notation)
- 定义：设 f 和 g 是从整数集合或实数集合到实数集合的函数，我们记 $f(x)$ 是 $O(g(x))$ 是说：存在正的常数 C 和 k ，使得当 $x > k$ ，有 $|f(x)| \leq C|g(x)|$ 。
- *常数 C 和 k 称为 $f(x)$ 是 $O(g(x))$ 的证据(witnesses)。
- *注意：如果存在 $f(x)$ 是 $O(g(x))$ 的证据 C 和 k ，那么这个证据有无穷多个。任取 $C' > C$, $k' > k$ ，当 $x > k' > k$ 时，有 $|f(x)| \leq C|g(x)| < C'|g(x)|$ 。

3.2 函数的增长率

- 例1：证明： $f(x) = x^2 + 2x + 1$ 是 $O(x^2)$ 。
- 解：当 $x > 1$ 时，有 $x < x^2, 1 < x^2$ 。因而有，当 $x > 1$ 时， $0 \leq x^2 + 2x + 1 \leq x^2 + 2x^2 + x^2 = 4x^2$ 。
- 故取 $C=4, k=1$ 作为证据，有 $f(x)$ 是 $O(x^2)$ 。
- 注意： $f(x)$ 是 $O(g(x))$ 也可以表示为 $f(x)=O(g(x))$ 或 $f(x) \in O(g(x))$
- $O(g(x))$ 表示所有是 $O(g(x))$ 的函数的集合。

3.2 函数的增长率

- 例2：证明： $7x^2$ 是 $O(x^3)$ 。
- 解：取 $k=7$ ， $C=1$ ，当 $x>7$ 时，有 $|7x^2| < C|x^3|$ ，故 $7x^2 = O(x^3)$ 。
- 例3：证明： n^2 不是 $O(n)$ 。
- 解：要证 n^2 不是 $O(n)$ ，就是要证不存在常数 $C>0$ ， $k>0$ ，使得只要 $n>k$ ，就有 $n^2 \leq Cn$ 。给定常数 $C>0$ ，要使 $n^2 \leq Cn$ ，就有 $n \leq C$ 。显然，对任意常数 $C>0$ ，只要取 $k>C$ ，当 $n>k>C$ 时，总有 $n \leq C$ 不成立。故 $n^2 \leq Cn$ 不成立。

3.2 函数的增长率

- 大O记号的一些重要结论
- *对任意多项式 f , f 是 $O(x^n)$, 其中 x^n 是 f 的最高次项
- 定理1: 设 $f(x) = a_n x^n + a_{n-1} x^{n-1} + \dots + a_1 x + a_0$, 其中 $a_0, a_1, \dots, a_n$ 都是实数。那么 $f(x)$ 是 $O(x^n)$ 。
- 证明: 如果 $x > 1$, 有
- $|f(x)| = |a_n x^n + a_{n-1} x^{n-1} + \dots + a_1 x + a_0|$
- $\leq |a_n| x^n + |a_{n-1}| x^{n-1} + \dots + |a_1| x + |a_0|$
- $= x^n (|a_n| + \frac{|a_{n-1}|}{x} + \dots + \frac{|a_1|}{x^{n-1}} + \frac{|a_0|}{x^n})$
- $\leq x^n (|a_n| + |a_{n-1}| + \dots + |a_0|)$
- $\leq C x^n$

其中, $C = |a_n| + |a_{n-1}| + \dots + |a_0|$, $x > k = 1$.

故 $f(x)$ 是 $O(x^n)$ 。

3.2 函数的增长率

- 例5：如何估计前 n 个正整数的和的大 O 函数。
- 解：因为 $1 + 2 + \cdots + n \leq n + n + \cdots + n = n^2$ ，取 $C=1$ ， $k=1$ ，当 $n \geq k = 1$ 时，有 $1 + 2 + \cdots + n \leq Cn^2$ ，故 $1 + 2 + \cdots + n$ 是 $O(n^2)$ 。
- 例6：给出 n 阶乘函数 $f(n)=n!$ 的大 O 函数。
- 解：因为 $f(n)=n!=1 \cdot 2 \cdot 3 \cdots n \leq n \cdot n \cdots n = n^n$ ，取 $k=1$ ， $C=1$ ，有 $f(n)=n!=O(n^n)$ 。另外， $\log n! \leq \log n^n = n \log n$ ，故取 $C=1$ ， $k=1$ ，有 $\log n! = O(n \log n)$ 。

3.2 函数的增长率

- 例7：已知对任意正整数 n ，有 $n < 2^n$ ，即 $n = O(2^n)$ 。利用这个不等式证明 $\log n = O(n)$ 。
- 解：取 $k=1$ ， $C=1$ ，有当 $n > k = 1$ 时，有 $n < 2^n$ ，故 $\log n < Cn = n$ 。
- 故 $\log n = O(n)$ 。
- *对于任意底数 b 的 $\log$ 函数，因为有
- $$\log_b n = \frac{\log n}{\log b} < \frac{n}{\log b}$$
- 取 $k=1$ ， $C = \frac{1}{\log b}$ ，有当 $n > k = 1$ 时， $\log_b n < Cn$ ，故 $\log_b n = O(n)$ 。

3.2 函数的增长率

- 一些常见的不同增长率的函数
- $1, \log n, n, n \log n, n^2, 2^n, n!$

3.2 函数组合的增长率

➤ 1. 函数的和

- 定理2: 假设 $f_1(x)$ 是 $O(g_1(x))$, $f_2(x)$ 是 $O(g_2(x))$, 那么 $(f_1 + f_2)(x)$ 是 $O(\max(|g_1(x)|, |g_2(x)|))$ 。
- 证明: 由定义知, 存在正的常数 C_1, C_2, k_1, k_2 , 使得, 当 $x > k_1, x > k_2$ 时, 有 $|f_1(x)| \leq C_1|g_1(x)|, |f_2(x)| \leq C_2|g_2(x)|$.
- 取 $k = \max(k_1, k_2)$, 当时 $x > k$, 有
- $|(f_1 + f_2)(x)| = |f_1(x) + f_2(x)|$
- $\leq |f_1(x)| + |f_2(x)|$
- $\leq C_1|g_1(x)| + C_2|g_2(x)|$
- $\leq C_1|g(x)| + C_2|g(x)|$
- $= (C_1 + C_2)|g(x)| = C|g(x)|$

其中, $C = C_1 + C_2, g(x) = \max(|g_1(x)|, |g_2(x)|)$.

3.2 函数组合的增长率

- **推论1:** 假设 $f_1(x)$ 和 $f_2(x)$ 都是 $O(g(x))$, 那么 $(f_1 + f_2)(x)$ 也是 $O(g(x))$ 。
- **2. 函数的积**
- **定理3:** 假设 $f_1(x)$ 是 $O(g_1(x))$, $f_2(x)$ 是 $O(g_2(x))$, 那么 $(f_1 f_2)(x)$ 是 $O(|g_1(x)| |g_2(x)|)$ 。

3.2 函数组合的增长率

- 证明：由定理知,存在 k_1, C_1 和 k_2, C_2 ,使得当 $x > k_1, x > k_2$ 时, 有 $|f_1(x)| \leq C_1 |g_1(x)|, |f_2(x)| \leq C_2 |g_2(x)|$ 。取 $k = \max(k_1, k_2)$, 当 $x > k$ 时, 有
- $|(f_1 f_2)(x)| = |f_1(x)| |f_2(x)|$
- $\leq C_1 |g_1(x)| \cdot C_2 |g_2(x)|$
- $= C_1 C_2 |g_1(x)| |g_2(x)|$
- $= C |g_1(x)| |g_2(x)|$.
- 其中 $C = C_1 C_2$ 。
- 注意:当 $g_1(x) \geq 0$ 和 $g_2(x) \geq 0$ 时, $O(|g_1(x)| |g_2(x)|) = O(g_1(x) g_2(x))$ 。

3.2 函数组合的增长率

- 例8：求以下函数 $f(n) = 3n \log(n!) + (n^2 + 3)\log n$ 的大O函数。
- 解：因为 $\log(n!) = O(n \log n)$, $3n = O(n)$, 故由定理3, 有 $3n \log n! = O(n^2 \log n)$. 而由定理2知, $n^2 + 3 = O(n^2)$. 再由定理3, 有 $(n^2 + 3)\log n = O(n^2 \log n)$. 再由定理2, 有 $f(n) = 3n \log(n!) + (n^2 + 3)\log n = O(n^2 \log n)$ 。

3.2 函数组合的增长率

- 例9：给出以下函数 $f(x) = (x + 1)\log(x^2 + 1) + 3x^2$ 的大O函数。
- 解：因为 $x^2 + 1 < 2x^2$ ($x > 1$ 时)，故 $\log(x^2 + 1) \leq \log(2x^2) = \log 2 + \log x^2 = \log 2 + 2\log x \leq 3\log x$ ($x > 2$)。所以 $\log(x^2 + 1) = O(\log x)$ 。又因为 $x+1=O(x)$ ，故
- $(x+1)\log(x^2 + 1)=O(x \log x)$ 。而 $3x^2 = O(x^2)$ 。由定理2，有
- $f(x) = O(\max(x \log x, x^2))=O(x^2)$ ，这因为当 $x>1$ 时，有 $x \log x < x^2$ 。

3.2 函数组合的增长率

➤ 4. 大 Ω 和大 Θ 记号

➤ 大 Ω 记号 (Big- Ω notation)

➤ 定义：设 f 和 g 是从整数集合或实数集合到实数集合的函数。我们说 $f(x)$ 是 $\Omega(g(x))$,是说, 存在正的常数 C 和 k , 使得, 当 $x > k$ 时, 有 $|f(x)| \geq C|g(x)|$ 。

➤ 例10：函数 $f(x) = 8x^3 + 5x^2 + 7$ 是 $\Omega(g(x))$ 的, 其中 $g(x) = x^3$ 。

➤ 由定义, 存在 $C=1$, $k=1$, 当 $x > k = 1$ 时, 有

➤ $|f(x)| = 8x^3 + 5x^2 + 7 \geq 8x^3 \geq Cx^3 = x^3$. 故 $f(x) = \Omega(x^3)$ 。

3.2 函数组合的增长率

- 大 Θ 记号 (Big- Θ notation)
- 定义：设 f 和 g 是从整数集合或实数集合到实数集合的函数。我们说 $f(x)$ 是 $\Theta(g(x))$ ，是说， $f(x)$ 是 $O(g(x))$ 且 $f(x)$ 是 $\Omega(g(x))$ 。
- 例11：（由例5）已证前 n 个正整数的和是 $O(n^2)$ 。问它是否具有阶 n^2 ？

3.2 函数组合的增长率

- 解：设 $f(n)=1+2+\cdots+n$.已知 $f(n)=O(n^2)$.取 $k=2$,当 $n>2$ 时,有
- $1+2+\cdots+n \geq \lfloor n/2 \rfloor + \left(\left\lfloor \frac{n}{2} \right\rfloor + 1\right) + \cdots + n$
- $\geq \lfloor n/2 \rfloor + \lfloor n/2 \rfloor + \cdots + \lfloor n/2 \rfloor$
- $= (n - \lfloor n/2 \rfloor + 1)\lfloor n/2 \rfloor$
- $\geq \left(\frac{n}{2}\right)\left(\frac{n}{2}\right)$
- $= n^2/4$
- $\geq Cn^2$
- 其中 $C=1/4$, 故 $f(n) = \Omega(n^2)$ 。又由已知 $f(n)=O(n^2)$, 有 $f(n)=\Theta(n^2)$ 。

3.2 函数组合的增长率

- **定理4:** 设 $f(x) = a_n x^n + a_{n-1} x^{n-1} + \cdots + a_1 x + a_0$, 其中 $a_0, a_1, \cdots, a_n$ 是实数且 $a_n \neq 0$ 。那么 $f(n) = \Theta(x^n)$ 。

3.3 算法的复杂性

➤ 1. 程序运行时间的测量

影响程序运行时间的因素：

- 程序的输入的长度；
- 编译程序生成目标代码的质量；
- 计算机指令的性质和速度；
- 算法的时间复杂性。

3.3 算法的复杂性

- 2. 评价算法运行时间的标准
- 运行时间作为输入长度的函数 $T(n)$ ，即对于长度为 n 的输入，算法执行运算的步数（操作的次数） $T(n)$.
- 最坏运行时间：
 - 算法对长度为 n 的任何输入的最长运行时间。
- 平均运行时间：
 - 即在“平均”输入下，算法的运行时间。通常我们假设给定长度的各种输入概率相同。平均运行时间是在这个假设下，运行时间的数学期望值。

3.3 算法的复杂性

- 为什么常用最坏运行时间来估计？
- 最坏运行时间是算法运行时间的上界，在实际问题中，算法的运行时间常常达到这个上界。平均运行时间难以计算，假设每个输入具有相同的概率有时没意义。平均运行时间常常与最坏运行时间有相同的数量级。

3.3 算法的复杂性

- 例1：求3.1节算法1(有穷长的整数序列中找最大的那个整数)的时间复杂度
- 解：我们用元素的比较次数作为估计时间复杂度的标准。
- 每扫描一个元素 a_i ， a_i 与max比较一次，然后判定 a_i 是否最后一个元素又作一次比较，故作了2次比较。共扫描 $n - 1$ 个元素（从 a_2 到 a_n ）。最后又作了一次比较，退出循环，故一共作了 $2(n - 1) + 1 = 2n - 1$ 次比较。故比较次数是 $\Theta(n)$ 。

3.3 算法的复杂性

- 例2：估计线性查找算法的时间复杂度。
- 解：每查一个元素， x 与 a_i 比较一次，要判定 a_i 是否表末尾，又要比较一次，故每查一个元素 a_i ，要比较2次。如果 $x = a_i$ ，则后面的元素就不用查了，故比较了 $2i+1$ 次，（退出循环后要作一次比较），因为我们要估计算法的最坏运行时间，即是 x 不在该序列中的情形，因而要查遍所有 a_i 。故要进行 $2n+2$ 次比较。所以，最坏运行时间是 $O(n)$ 。

➤ P6

3.3 算法的复杂性

- 理解算法的时间复杂度
- 当一个算法的时间复杂度为 $O(f(n))$, 其中 $f(n)$ 是一个多项式, 那么, 这个算法称为“好算法” (good algorithm) 或者 “有效算法” (efficient algorithm)。
- 如果一个算法的时间复杂度是 $\Theta(f(n))$, 而 $f(n)$ 是指数函数或阶乘函数或更大, 那么, 这个算法称为“坏算法” (bad algorithm) 。
- 旅行商问题

3.3 算法的复杂性

- 易处理的问题(tractable problems)和难处理的问题(intractable problems)
- 如果一个问题有最坏情况是多项式时间的求解算法，那么这个问题称为易处理（易解）的；
- 如果一个问题没有最坏情况是多项式时间的求解算法，那么这个问题称为难处理（难解）的。
- P, NP, NP-完全问题

作业10

- 1. 描述一个算法，该算法输入 n 个不同的整数 $a_1, a_2, \dots, a_n$ ，并找出其中最大的偶数 a_i 的位置 i ，若其中没有偶数，则 i 为0。
- 2. 找一个最小的整数 n ，使得 $f(x)$ 是 $O(x^n)$.
 - (1) $f(x) = 2x^3 + x^2 \log x$
 - (2) $f(x) = (x^4 + x^2 + 1)/(x^3 + 1)$
- 3. 给出以下函数 $f(n)$ 的大 O 函数 $g(n)$ ，使得 $f(n) = O(g(n))$ 。
 - $f(n) = (n^3 + n^2 \log n)(\log n + 1) + (17 \log n + 19)(n^3 + 2)$
 - 4. 分析二分查找算法的时间复杂度。

3.4 整数与整除

- 3.4.1 整除
- 3.4.2 除算法
- 3.4.3 模算术 (Modular Arithmetic)
- 3.4.4 同余的应用

3.4.1 整除 (Division)

- 整除
- 定义：设 a 和 b 是整数且 $a \neq 0$ ，我们说 a 整除 b (a divides b)，是说，存在整数 c 使得 $b=ac$ 。当 a 整除 b ，我们说 a 是 b 的因子(factor)并且 b 是 a 的倍数(multiple of a)，记作 $a|b$ 。若 a 不能整除 b ，记作 $a \nmid b$ 。
- 例1：判定是否有 $3|7$ 和 $3|12$ 。
- 解：因为 $7/3$ 不是整数，所以 $3 \nmid 7$ ，而 $\frac{12}{3} = 4$ ，故 $3|12$ 。

3.4.1 整除

- 例2：设 n 和 d 是正整数。问有多少个不超过 n 的正整数可以被 d 整除？
- 解：能被 d 整除的正整数具有形式 dk ，其中 k 是正整数，不超过 n 可以被 d 整除的正整数的个数，就是满足 $0 \leq dk \leq n$ 的 k 的个数，也就是满足 $0 < k \leq n/d$ 的 k 的个数，即 $\lfloor n/d \rfloor$ 。

3.4.1 整除

➤ 整除的性质

➤ 定理1：设 a, b, c 是整数，那么

➤ (i)如果 $a|b$ 且 $a|c$ ，那么 $a|(b + c)$ ；

➤ (ii)如果 $a|b$ ，那么 $a|bc$ 对所有整数 c 成立；

➤ (iii)如果 $a|b$ 且 $b|c$ ，那么 $a|c$ 。

➤ 证明：(i) 假设 $a|b$ 且 $a|c$ ，那么由定义知，存在整数 s 和 t ，使得 $b=as$ ， $c=at$ ，故 $b+c=as+at=a(s+t)$ 。
从而 $a|(b + c)$ 。

(ii)和(iii)的证明类似。

3.4.1 整除

- **推理1:** 如果 a, b, c 是整数且 $a|b$ 和 $a|c$, 那么 $a|(mb + nc)$, 其中 m, n 是任意整数。

3.4.2 除算法

- 定理2：（除算法）设 a 是整数， d 是一个正整数，那么存在唯一的整数 q 和 r 且 $0 \leq r < d$ ，有 $a = dq + r$ 。
- 定义：在除算法中， d 称为除数(divisor)， a 称为被除数(dividend)， q 称为商(quotient)， r 称为余数(remainder)。商和余数记为： $q = a \text{ div } d$ ， $r = a \text{ mod } d$ 。

3.4.2 除算法

- 例3：101除以11，商和余数是多少？
- 解： $101 = 11 \cdot 9 + 2$ ，商是9，余数是2。
- 例4：-11除以3，商和余数是多少？
- 解： $-11 = 3 \cdot (-4) + 1$ ，故商是-4，余数是1。
- 注意：余数必须是非负数，不能写成 $-11 = 3 \cdot (-3) - 2$ 。

3.4.3 模算术 (Modular Arithmetic)



- 模和余数
- 设 a 和 b 是整数， m 是正整数，那么 a 和 b 模 m 同余 (a is congruent to b modulo m), 是说 m 整除 $a - b$, 记为 $a \equiv b(\text{mod } m)$ 。若 a 和 b 模 m 不同余, 记为 $a \not\equiv b(\text{mod } m)$ 。
- 模 m 同余的充要条件
- 定理3: 设 a 和 b 是整数， m 是正整数。那么 $a \equiv b(\text{mod } m)$ 当且仅当 $a \text{ mod } m = b \text{ mod } m$ 。

3.4.3 模算术

- 例5：判定17和5是否模6同余？24和14是否模6同余？
- 解：因为 $17 \bmod 6 = 5 = 5 \bmod 6$, 故 $17 \equiv 5(\bmod 6)$ 。
- 又因为 $24 - 14 = 10$ 不能被6整除，故 $24 \not\equiv 14(\bmod 6)$ 。

3.4.3 模算术

- 定理4：设 m 是正整数。整数 a 和 b 模 m 同余当且仅当存在整数 k 使得 $a = b + km$ 。
- 证明：设 $a \equiv b \pmod{m}$ 。那么 $m \mid (a - b)$ 。故存在整数 k ，使得 $(a - b) = km$ ，从而有 $a = b + km$ 。反过来，假设 $a = b + km$ ，对某个整数 k 成立，那么 $a - b = km$ ，从而有 $m \mid (a - b)$ 。故 $a \equiv b \pmod{m}$

3.4.3 模算术

- 模运算的性质
- 定理5：设 m 是正整数。如果 $a \equiv b \pmod{m}$ 且 $c \equiv d \pmod{m}$ ，那么 $a + c \equiv b + d \pmod{m}$ 且 $ac \equiv bd \pmod{m}$ 。
- 证明：因为 $a \equiv b \pmod{m}$ 和 $c \equiv d \pmod{m}$ ，因而存在整数 s 和 t ，使得 $b = a + sm$ 且 $d = c + tm$ 。因此
- $b + d = (a + sm) + (c + tm) = (a + c) + m(s + t)$
- 且 $bd = (a + sm)(c + tm) = ac + m(at + cs + stm)$
- 因此 $a + c \equiv b + d \pmod{m}$ 且 $ac \equiv bd \pmod{m}$ 。

3.4.3 模算术

- 例6：因为 $7 \equiv 2(\text{mod } 5)$ 且 $11 \equiv 1(\text{mod } 5)$ ，由定理知 $18 = 7 + 11 \equiv 2 + 1 = 3(\text{mod } 5)$ 且 $77 = 7 \cdot 11 \equiv 2 \cdot 1 = 2(\text{mod } 5)$ 。

3.4.3 模算术

- **推论2:** 设 m 是正整数, 且 a 和 b 是整数。那么
- $(a + b) \bmod m = ((a \bmod m) + (b \bmod m)) \bmod m$
- 且 $ab \bmod m = ((a \bmod m)(b \bmod m)) \bmod m$ 。

- **证明:** 因为 $a \equiv (a \bmod m) \bmod m$, $b \equiv (b \bmod m) \bmod m$, 由定理5知,
 $(a + b) \bmod m = ((a \bmod m) + (b \bmod m)) \bmod m$
- 且 $ab \bmod m = ((a \bmod m)(b \bmod m)) \bmod m$ 。

3.4.4 同余的应用

- 凯撒密码 (密码学(Cryptology))
- 将英文字母表中所有字母向前移3个位置，最前面的3个字母移到最后面3个位置。用字母表
- D,E,F,G,H, I, J,K,L,M,N,O,P, Q,R,S,T,U,V,W, X,Y,Z,A,B,C 对应
- A,B,C,D,E,F,G,H,I, J, K, L,M,N,O,P,Q,R,S,T, U,V,W,X,Y,Z,
- 进行编码。

3.4.4 同余的应用

- 例如： MEET YOU IN THE PARK 编码为
- PHHW BRX LQ WKH SDUN
- 假设用0, 1, 2, ..., 25给26个字母编号，凯撒密码用公式表示为： $f(p) = (p+3) \bmod 26$, $p \in \{0, 1, 2, \dots, 25\}$. $f(p)$ 为加密函数(encryption function).
- 解密函数(decryption function)为： $f^{-1}(p) = (p-3) \bmod 26$ 。

3.4.4 同余的应用

- 凯撒密码可以推广到更一般的形式:
- $f(p) = (ap + b) \bmod 26,$
- 其中 a, b 为整数, 经过选取 a, b , 使得 $f(p)$ 为 $\{0, 1, \dots, 25\} \rightarrow \{0, 1, \dots, 25\}$ 的双射函数。

3.5 素数和最大公因子

- 3.5.1 素数 (primes)
- 3.5.2 关于素数的公开问题与猜想
- 3.5.3 最大公因子和最小公倍数

3.5.1 素数 (primes)

- 定义：一个大于1的正整数 p 被称为素数，是说， p 只有正因子 p 和1。一个大于1的正整数 q 如果不是素数，则 q 被称为合数(composite)。
- 例1：整数7是素数，因为它只有因子7和1，而整数9是合数，因为它有因子3且 $3 \neq 1, 9$ 。
- 定理1（算术基本定理）：每一个大于1的正整数可以表示成一个素数，或可以表示成两个或更多的素数的乘积，其中这些素因子可以写成非递减的顺序。

3.5.1 素数

- 例2：给出100，641，999和1024的素因子分解
- 解： $100 = 2 \cdot 2 \cdot 5 \cdot 5 = 2^2 \cdot 5^2$
- $641 = 641$
- $999 = 3 \cdot 3 \cdot 3 \cdot 37 = 3^3 \cdot 37$
- $1024 = 2 \cdot 2 \cdot 2 \cdot 2 \cdot 2 \cdot 2 \cdot 2 \cdot 2 \cdot 2 \cdot 2 = 2^{10}$ 。

3.5.1 素数

- 定理2：如果 n 是一个合数，那么 n 有一个小于或等于 $\sqrt{n}$ 的素因子。
- 证明：假设 n 是合数，那么 n 有一个因子 a ， $1 < a < n$ ，并且 n 可以写成 $n=ab$ ，其中 b 是大于1的正整数。这时，有 $a \leq \sqrt{n}$ 或 $b \leq \sqrt{n}$ 。假若不然，有 $a > \sqrt{n}$ 且 $b > \sqrt{n}$ ，那么 $n=ab > \sqrt{n}\sqrt{n} = n$ ，矛盾。故有 $a \leq \sqrt{n}$ 或 $b \leq \sqrt{n}$ 。因为 a 和 b 都是 n 的因子，故 n 有一个不大于 $\sqrt{n}$ 的因子 a （或 b ）。而 a （或 b ）要么是一个素数，要么由算术基本定理， a （或 b ）有一个小于 a （或 b ）的素因子，在两种情况下， n 都有一个小于或等于 $\sqrt{n}$ 的素因子。

3.5.1 素数

- 例3：证明101是素数。
- 证明：只要证101没有小于或等于 $\sqrt{101}$ 的素因子即可。因为小于或等于 $\sqrt{101}$ 的素数只有2,3,5,7，而2,3,5,7都不是101的因子，故101是素数。

3.5.1 素数

- 求整数 n 的素数因子分解
- 对于任一整数 n ，从最小的素数2开始，用从小到不大超过 $\sqrt{n}$ 的素数去除 n ，若求得一个素因子 p ，那么再对 n/p ，从 p 开始从小到大求不超过 $\sqrt{n/p}$ 的素因子。如果又求得素因子 q ，再对 $n/(pq)$ ，从 q 开始，从小到大求不超过 $\sqrt{n/(pq)}$ 的素因子，如此进行下去。最后可求出 n 的所有素因子。

3.5.1 素数

- 例4：求7007的素因子分解。
- 解：从2开始，用从小到大的素数去除7007。
2,3,5都不能整除7007. 而7可以整除7007. 7是第一个素因子。再从7开始用素数去除 $\frac{7007}{7} = 1001$, 这时有 $7|1001$ 。故7是第2个素因子。再从7开始用素数去除 $\frac{1001}{7} = 143$, 这时找到 $11|143$, 故11是第3个素因子。再从11开始用素数去除 $\frac{143}{11} = 13$, 求出13是最后一个素因子。故7007的素因子分解为：
 $7007=7 \cdot 7 \cdot 11 \cdot 13 = 7^2 \cdot 11 \cdot 13$ 。

3.5.1 素数

- 素数的无穷性 (The infinitude of primes)
- 定理3: 存在无穷多个素数。
- 证明: 反证法。假设只有有穷多个素数, 设 $p_1, p_2, \dots, p_n$ 是所有的素数, 那么令 $Q = p_1 p_2 \cdots p_n + 1$ 。由算术基本定理, Q 应能表示成 $p_1, p_2, \dots, p_n$ 中某些素数的因子分解。
- 但对任意 p_i ($1 \leq i \leq n$), $p_i \nmid Q$ 。这因为, 假若 $p_i \mid Q$, 又有 $p_i \mid p_1 p_2 \cdots p_n$, 故有 $p_i \mid (Q - p_1 p_2 \cdots p_n)$ 。即 $p_i \mid 1$, 矛盾。故 Q 是一个素数且 $Q \neq p_i, i=1, 2, \dots, n$ 。这与 $p_1, p_2, \dots, p_n$ 是所有素数的假设矛盾。

3.5.1 素数

- *因为素数有无穷多个，人们试图列出尽可能大的素数（在密码学中有用）。已知的最大素数具有这样的表达式： $2^p - 1$ ，其中 p 是一个素数，这种素数称为梅森数(Mersenn primes)。但并不是所有具有表达式 $2^p - 1$ 的数都是素数。
- 例5： $2^2 - 1 = 3$ ， $2^3 - 1 = 7$ ， $2^5 - 1 = 31$ 都是梅森数，但 $2^{11} - 1 = 2047$ 不是梅森数，因为 $2047 = 23 \cdot 89$ 。
- *目前已知的最大的梅森数是 $2^{30,402,457} - 1$ ，这是一个9百万位10进位的数字。

3.5.1 素数

- 定理4（素数定理）：不超过 x 的素数个数的频率是 $x/\ln x$ （当 x 趋于无穷）。
- *其中 $\ln x$ 是 x 的自然对数。

3.5.2 关于素数的公开问题与猜想



- 哥德巴赫猜想：
- 任何一个大于5的奇数可以表示成3个素数的和。
- 等价命题：任何一个大于2的偶数可以表示成2个素数的和。
- 目前已知的最好结果：
- (Ramare1995)：任何一个大于2的偶数可以表示成至多6个素数的和。
- (陈景润1966)：任何一个大于2的偶数可以表示成两个素数之和或一个素数和两个素数的乘积之和。

3.5.2 关于素数的公开问题与猜想



- 孪生素数猜想:
- 有无穷多个孪生素数 p 和 $p+2$ 。
- 例如：3和5， 5和7， 11和13， 17和19， 4967和4969等。

3.5.3 最大公因子和最小公倍数

- 最大公因子 (greatest common divisors)
- 定义：设 a 和 b 都是非零整数。最大整数 d 满足： $d|a$ 且 $d|b$ ，称为 a 和 b 的最大公因子，记作 $d=\gcd(a, b)$
- 例10：24和36的最大公因子是什么？
- 解：24和36的正的公因子有1,2,3,4,6和12。因此 $\gcd(24,36)=12$ 。
- 例11：17和22的最大公因子是什么？
- 解：17和22的公因子只有1。故 $\gcd(17,22)=1$ 。

3.5.3 最大公因子和最小公倍数

- 两个整数互素
- 定义：如果两个整数 a, b ，满足 $\gcd(a, b)=1$ ，那么称 a 和 b 互素(relatively prime)。
- 例如：例11中，17和22互素。
- 定义：整数 $a_1, a_2, \dots, a_n$ 两两互素是说 $\gcd(a_i, a_j) = 1 (1 \leq i < j \leq n)$ 。
- 例13：10, 17, 21两两互素。

3.5.3 最大公因子和最小公倍数

- 求最大公因子的方法
- 已知两个整数 a 和 b ，由算术基本定理， a, b 可表示成 $a = p_1^{a_1} p_2^{a_2} \cdots p_n^{a_n}$, $b = p_1^{b_1} p_2^{b_2} \cdots p_n^{b_n}$
- 其中 a_i 和 b_j 可能为0.
- 那么 $\gcd(a, b) = p_1^{\min(a_1, b_1)} p_2^{\min(a_2, b_2)} \cdots p_n^{\min(a_n, b_n)}$
- *首先，上式右边的数可以整除 a 和 b ，其次，更大的数不能整除 a 或 b 。

3.5.3 最大公因子和最小公倍数

- 例14：求120和500的最大公因子。
- 解： $120=2^3 \cdot 3 \cdot 5$, $500 = 2^2 \cdot 5^3$
- $\gcd(120,500) = 2^{\min(3,2)} \cdot 3^{\min(1,0)} \cdot 5^{\min(1,3)}$
 $= 2^2 \cdot 3^0 \cdot 5^1 = 20$

3.5.3 最大公因子和最小公倍数

- 最小公倍数(least common multiple)
- 定义：正整数a和b的最小公倍数是能够被a和b整除的最小正整数。记作：lcm(a, b).
- 假设

$$a = p_1^{a_1} p_2^{a_2} \cdots p_n^{a_n}, \quad b = p_1^{b_1} p_2^{b_2} \cdots p_n^{b_n}$$

- 那么

- $$\text{lcm}(a, b) = p_1^{\max(a_1, b_1)} p_2^{\max(a_2, b_2)} \cdots p_n^{\max(a_n, b_n)}$$

3.5.3 最大公因子和最小公倍数

- 例15：求 $2^3 \cdot 3^5 \cdot 7^2$ 和 $2^4 \cdot 3^3$ 的最小公倍数。
- 解：
- $\text{lcm}(2^3 \cdot 3^5 \cdot 7^2, 2^4 \cdot 3^3) = 2^{\max(3,4)} \cdot 3^{\max(5,3)} \cdot 7^{\max(2,0)}$
- $= 2^4 \cdot 3^5 \cdot 7^2$ 。
- **定理5：** 设 a, b 是正整数，那么 $a \cdot b = \text{gcd}(a, b) \text{lcm}(a, b)$ 。

作业11

- 1. 证明：如果 a, b 是整数，并且 $a|b$ 且 $b|a$ ，那么 $a=b$ 或 $a=-b$ 。
- 证明：如果 $a \equiv b \pmod{m}$ 且 $c \equiv d \pmod{m}$ ，其中 a, b, c 是整数， m 是正整数，那么 $(a - c) \equiv (b - d) \pmod{m}$ 。
- 找出以下数字的素因子分解：
 - (1) 126 (2) 729 (3) 143
- 4. 求以下数对的最大公因子：
 - (1) $2 \cdot 3 \cdot 5 \cdot 7 \cdot 11 \cdot 13$; $2^{11} \cdot 3^9 \cdot 11 \cdot 17^{14}$
 - (2) 66; 126

3.6 整数和算法

- 3.6.1 整数的表示
- 3.6.2 整数运算的算法
- 3.6.3 指数取模运算
- 3.6.4 欧几里德算法

3.6.1 整数的表示

- 整数965可以表示成十进制数(以10为基(base)):
 $9 \cdot 10^2 + 6 \cdot 10 + 5$,也可以表示成以2为基的数(二进制数),或以8为基的数(八进制数)等等。
- 一个整数 n 可以表示成以任意大于1的正整数 b 为基的数。
- **定理1:** 设 b 是大于1的正整数。如果 n 是任一正整数,那么 n 可以唯一地表示成:

$$n = a_k b^k + a_{k-1} b^{k-1} + \cdots + a_1 b + a_0$$

- 其中, k 是一非负整数, $a_0, a_1, \cdots, a_k$ 是小于 b 的非负整数且 $a_k \neq 0$ 。

3.6.1 整数的表示

- *定理1中的 n 的表示称为 n 的以 b 为基的展开式，记作 $(a_k a_{k-1} \cdots a_1 a_0)_b$ 。
- 例如： $(245)_8 = 2 \cdot 8^2 + 4 \cdot 8 + 5$.
- *以2为基的展开式称为二进制展开式(binary expansion)。
- 例1：二进制展开式 $(1\ 0101\ 1111)_2$ 的十进制数是什么？
- 解： $(1\ 0101\ 1111)_2 = 1 \cdot 2^8 + 0 \cdot 2^7 + 1 \cdot 2^6 + 0 \cdot 2^5 + 1 \cdot 2^4 + 1 \cdot 2^3 + 1 \cdot 2^2 + 1 \cdot 2^1 + 1 \cdot 2^0 = 351$

3.6.1 整数的表示

- 例3：将十进制数 $(12345)_{10}$ 转化为八进制展开式 (octal expansion).
- 解： $12345 = 8 \cdot 1543 + 1$
- $1543 = 8 \cdot 192 + 7$
- $192 = 8 \cdot 24 + 0$
- $24 = 8 \cdot 3 + 0$
- $3 = 8 \cdot 0 + 3$
- 于是： $(12345)_{10} = (30071)_8$ 。

3.6.1 整数的表示

- *十六进制数的 a_i 取值0, 1, 2, 3, 4, 5, 6, 7, 8, 9, A(10), B(11), C(12), D(13), E(14), F(15)。
- 例4：求十进制数 $(177130)_{10}$ 的十六进制展开式(hexadecimal expansion)。
- 解： $177130 = 16 \cdot 11070 + 10$
- $11070 = 16 \cdot 691 + 14$
- $691 = 16 \cdot 43 + 3$
- $43 = 16 \cdot 2 + 11$
- $2 = 16 \cdot 0 + 2$

于是： $(177130)_{10} = (2B3EA)_{16}$ 。

3.6.1 整数的表示

➤ 例5：求十进制数 $(241)_{10}$ 的二进制展开式。

➤ 解： $241 = 2 \cdot 120 + 1$

➤ $120 = 2 \cdot 60 + 0$

➤ $60 = 2 \cdot 30 + 0$

➤ $30 = 2 \cdot 15 + 0$

➤ $15 = 2 \cdot 7 + 1$

➤ $7 = 2 \cdot 3 + 1$

➤ $3 = 2 \cdot 1 + 1$

➤ $1 = 2 \cdot 0 + 1$

于是： $(241)_{10} = (1111\ 0001)_2$ 。

3.6.1 整数的表示

- 算法1: (10进制转换为b进制展开式的算法)
- PROCEDURE base b expansion (n: positive integer);
- $q := n;$
- $k := 0;$
- WHILE $q \neq 0$ DO
- BEGIN
- $a_k := q \bmod b;$
- $q := \lfloor q/b \rfloor;$
- $k := k+1;$
- END;
- {the base b expansion of n is $(a_{k-1}a_{k-2} \cdots a_1a_0)_b$ }.

3.6.1 整数的表示

- *二进制数转换为八进制、十六进制展开式。
- 例6: $(11\ 1110\ 1011\ 1100)_2$ 转换为16进制展开式和8进制展开式。
- 解: (1) 转换为16进制展开式: 从个位起, 将2进制数每4位分为一个区段, 得: 0011, 1110, 1011, 1100, 再将每个区段转换为16进制数, 得: 3, E, B, C, 故 $(11\ 1110\ 1011\ 1100)_2 = (3EBC)_{16}$
- (2) 转换为8进制展开式: 从个位起, 将2进制数每3位分为一个区段, 得: 011, 111, 010, 111, 100, 再将每个区段转换为8进制数, 得: 3, 7, 2, 7, 4 故 $(11\ 1110\ 1011\ 1100)_2 = (37274)_8$ 。

3.6.2 整数运算的算法

- 1. 加法:
- 例7: 求二进制数 $(1110)_2$ 和 $(1011)_2$ 的和。
- 解: 从个位加起, $a = (a_3a_2a_1a_0)_2$, $b = (b_3b_2b_1b_0)_2$.
- $a_0 + b_0 = 0 + 1 = 0 \cdot 2 + 1$, 有 $c_0 = 0, s_0 = 1$
 $a_1 + b_1 + c_0 = 1 + 1 + 0 = 1 \cdot 2 + 0$, 有 $c_1 = 1, s_1 = 0$
 $a_2 + b_2 + c_1 = 1 + 0 + 1 = 1 \cdot 2 + 0$, 有 $c_2 = 1, s_2 = 0$
 $a_3 + b_3 + c_2 = 1 + 1 + 1 = 1 \cdot 2 + 1$, 有 $c_3 = 1, s_3 = 1$
- 最后 $s_4 = c_3 = 1$, 得 $a + b = (s_4s_3s_2s_1s_0)_2 = (11001)_2$

3.6.2 整数运算的算法

- 算法2: (二进制加法)
- PROCEDURE add (a, b: positive integers);
- {the binary expansions of a and b are $(a_{n-1}a_{n-2} \cdots a_1a_0)_2$ and $(b_{n-1}b_{n-2} \cdots b_1b_0)_2$ }
- c := 0;
- FOR j := 0 TO n - 1 DO
- BEGIN
- d := $\lfloor (a_j + b_j + c)/2 \rfloor$;
- s_j := a_j + b_j + c - 2d;
- c := d;
- END;
- s_n := c ; {the binary expansion of the sum is $(s_ns_{n-1} \cdots s_0)_2$ }.

3.6.2 整数运算的算法

➤ 2. 乘法:

➤ 设 $a = a_k 2^k + a_{k-1} 2^{k-1} + \dots + a_1 2 + a_0$

➤ $b = b_{n-1} 2^{n-1} + b_{n-2} 2^{n-2} + \dots + b_1 2 + b_0$

➤ 那么 $ab = a(b_0 2^0 + b_1 2^1 + \dots + b_{n-1} 2^{n-1})$

➤ $= a(b_0 2^0) + a(b_1 2^1) + \dots + a(b_{n-1} 2^{n-1})$

3.6.2 整数运算的算法

- 例9：求二进制数 $a = (110)_2$ 和 $b = (101)_2$ 的积。
- 解： $ab_0 \cdot 2^0 = (110)_2 \cdot 1 \cdot 2^0 = (110)_2$
- $ab_1 \cdot 2^1 = (110)_2 \cdot 0 \cdot 2^1 = (0000)_2$
- $ab_2 \cdot 2^2 = (110)_2 \cdot 1 \cdot 2^2 = (11000)_2$
- 然后求 $(110)_2, (0000)_2, (11000)_2$ 三个数的和，得
 $ab = (11110)_2$.

3.6.2 整数运算的算法

- 3. 除法
- 算法4：（除和取模算法）
- PROCEDURE division (a: integer; d: positive integer);
- $q := 0;$
- $r := |a|;$
- WHILE $r \geq d$ DO
- BEGIN
- $r := r - d;$
- $q := q + 1;$
- END;

3.6.2 整数运算的算法

- **IF** $a < 0$ *and* $r > 0$ **THEN**
- **BEGIN**
- $r := d - r;$
- $q := -(q + 1);$
- **END;**
- $\{q = a \text{ div } d \text{ is the quotient, } r = a \text{ mod } d \text{ is the remainder}\}.$

3.6.3 指数取模运算

- *在密码学中，要求能有效地计算 $b^n \bmod m$ ，其中 b, n, m 都是大的整数。
- 设 $n = (a_{k-1}a_{k-2} \cdots a_0)_2$
- $b^n = b^{a_{k-1}2^{k-1} + \cdots + a_1 \cdot 2 + a_0}$
- $= b^{a_{k-1} \cdot 2^{k-1}} b^{a_{k-2} \cdot 2^{k-2}} \cdots b^{a_1 \cdot 2} b^{a_0}$
- 要计算 b^n ，我们先计算 $b, b^2, (b^2)^2 = b^4, (b^4)^2 = b^8, \dots, b^{2^k}$ ，然后把所有这样的项 b^{2^j} 乘起来，其中 $a_j = 1$ 。

3.6.3 指数取模运算

- 例如：要计算 3^{11} ，已知 $11=(1011)_2$.
- 因此， $3^{11} = 3^{1 \cdot 2^3 + 0 \cdot 2^2 + 1 \cdot 2^1 + 1} = 3^8 \cdot 3^2 \cdot 3^1$ ，我们先算： $3, 3^2 = 9, 3^4 = 9^2 = 81, 3^8 = 81^2 = 6561$ ，然后 $3^{11} = 3^8 \cdot 3^2 \cdot 3^1 = 6561 \cdot 9 \cdot 3 = 177,147$.
- 因为是取模运算，我们算 $b \bmod m$ ， $b^2 \bmod m, b^4 \bmod m, \dots, b^{2^{k-1}} \bmod m$ ，再对所有 $a_j = 1$ 的项，将 $b^{2^j} \bmod m$ 乘在一起，再取 $\bmod m$ 运算。

3.6.3 指数取模运算

- 算法5: (指数取模运算)
- PROCEDURE modular exponentiation (b : integer;
- $n = (a_{k-1}a_{k-2} \cdots a_1a_0)_2$, m : positive integer);
- $x := 1$;
- $power := b \bmod m$;
- FOR $i := 0$ TO $k - 1$ DO
- BEGIN
- IF $a_i = 1$ THEN $x := (x \cdot power) \bmod m$;
- $power := (power \cdot power) \bmod m$;
- END;
- { x equals $b^n \bmod m$ }

3.6.4 欧几里德算法

- 直接从整数的素因子分解式计算两个整数的最大公约数效率低（寻找素因子分解式耗时）
- 例子：求 $\gcd(91, 287)$
- 解：首先，用小的那个数91去除大的数287，得 $287 = 91 \cdot 3 + 14$ ，因此，287和91的任何公因子一定能整除 $287 - 91 \cdot 3 = 14$ 。故 $\gcd(91, 287) = \gcd(14, 91)$ 。
- 再用14去除91，得 $91 = 14 \cdot 6 + 7$ ，91和14的任何公因子一定能整除 $91 - 14 \cdot 6 = 7$ 。故 $\gcd(14, 91) = \gcd(7, 14)$ 。
- 再用7去除14，得 $14 = 7 \cdot 2$ ，故7就是7和14的公因子，故 $\gcd(7, 14) = 7$ ，从而 $\gcd(91, 287) = \gcd(14, 91) = \gcd(7, 14) = 7$ 。

3.6.4 欧几里德算法

- 引理1：设 $a=bq + r$ ，其中 a, b, q 和 r 都是整数，那么 $\gcd(a, b) = \gcd(b, r)$ 。
- 证明：如果我们能证明 a 和 b 的公因子就是 b 和 r 的公因子，那么就证明了 $\gcd(a, b) = \gcd(b, r)$ ，因为这两对数有同样的最大公因子。
- 假设 d 整除 a 和 b ，那么 d 也整除 $a - bq = r$ （由3.4节定理1得到）。因此， a 和 b 的任何公因子也是 b 和 r 的公因子。
- 类似地，假设 d 整除 b 和 r ，那么 d 也整除 $bq + r = a$ ，因此， b 和 r 的任何公因子也是 a 和 b 的公因子。

结果有 $\gcd(a, b) = \gcd(b, r)$ 。

3.6.4 欧几里德算法

- 设 a, b 是正整数且 $a \geq b$, 令 $r_0 = a, r_1 = b$, 反复应用除算法, 我们得到:
- $r_0 = r_1 q_1 + r_2 \quad 0 < r_2 < r_1$
- $r_1 = r_2 q_2 + r_3 \quad 0 < r_3 < r_2$
- $\vdots$
- $r_{n-2} = r_{n-1} q_{n-1} + r_n \quad 0 < r_n < r_{n-1}$
- $r_{n-1} = r_n q_n$
- 因为 $a = r_0 > r_1 > r_2 > \cdots \geq 0$, 该算法必然终止。反复应用引理1, 有 $\gcd(a, b) = \gcd(r_0, r_1) = \gcd(r_1, r_2) = \cdots = \gcd(r_{n-2}, r_{n-1}) = \gcd(r_{n-1}, r_n) = r_n$.

3.6.4 欧几里德算法

- 算法6：（欧几里德算法）
- PROCEDURE gcd (a, b: positive integers);
- $x := a;$
- $y := b;$
- WHILE $y \neq 0$ DO
- BEGIN
- $r := x \bmod y;$
- $x := y;$
- $y := r;$
- END;
- {gcd(a, b) is x}

3.7 数论的应用

- 3.7.1 一些有用的结果
- 3.7.2 线性同余式
- 3.7.3 中国剩余定理
- 3.7.4 费马小定理

3.7.1 一些有用的结果

- 定理1：设 a 和 b 是正整数，那么存在整数 s 和 t 使得 $\gcd(a,b)=sa+tb$ 。
- *我们不给出正式的证明，但通过用欧几里德算法求最大公因子的例子说明。
- 例1：将 $\gcd(252, 198)=18$ 表示成252和198的线性组合(linear combination)。

3.7.1 一些有用的结果

- 解：用欧几里德算法求 $\gcd(252, 198)=18$ 的过程如下：
- $252 = 1 \cdot 198 + 54$ (1)
- $198 = 3 \cdot 54 + 36$ (2)
- $54 = 1 \cdot 36 + 18$ (3)
- $36 = 2 \cdot 18$ (4)
- 由(3)式，有 $18 = 54 - 1 \cdot 36$ ，由(2)式 $36 = 198 - 3 \cdot 54$.
代入上式，得
- $18 = 54 - 1 \cdot (198 - 3 \cdot 54) = 4 \cdot 54 - 1 \cdot 198$ (5)
- 再由(1)式，有 $54 = 252 - 1 \cdot 198$ ，代入(5)式，得
- $18 = 4 \cdot (252 - 1 \cdot 198) - 1 \cdot 198 = 4 \cdot 252 - 5 \cdot 198$
- 这就是我们要求的解。

3.7.1 一些有用的结果

- 引理1：如果 a, b, c 是正整数并且 $\gcd(a, b) = 1$ 和 $a | bc$, 那么 $a | c$ 。
- 证明：因为 $\gcd(a, b) = 1$, 由定理1, 存在整数 s 和 t , 有
$$sa + tb = 1$$
- 上式两边同时乘以 c , 有 $sac + tbc = c$.
- 由3.4节定理1, 由该定理中(ii), 有 $a | tbc$, 又因为 $a | sac$ 和 $a | tbc$, 由该定理(i), 有 $a | (sac + tbc)$, 即 $a | c$.

3.7.1 一些有用的结果

- 引理2：如果 p 是素数且 $p|a_1a_2\cdots a_n$ ，其中每个 a_i 是整数，那么 $p|a_i$ 对某个 i 成立。
- *我们现在证明整数的素数因子分解是唯一的，即3.5节定理1中表示的唯一性。

3.7.1 一些有用的结果

- 证明整数的素数因子分解是唯一的
- 证明：用反证法。假设正整数 n 可以表示成两组不同的素数的乘积，即 $n = p_1 p_2 \cdots p_s$ 且 $n = q_1 q_2 \cdots q_t$ ，其中每个 p_i 和 q_j 都是素数，并且 $p_1 \leq p_2 \leq \cdots \leq p_s$ 和 $q_1 \leq q_2 \leq \cdots \leq q_t$ 。
- 由于 $n = p_1 p_2 \cdots p_s = q_1 q_2 \cdots q_t$ ，我们删除两组素数中共同的素数（公因子），有 $p_{i_1} p_{i_2} \cdots p_{i_u} = q_{j_1} q_{j_2} \cdots q_{j_v}$ 。
- 这时，等式两边没有共同的素数且 u 和 v 是正整数。由引理2，左边的 p_{i_1} 整除 q_{j_k} 对某个 k 成立，但任何一个素数不能整除另一个素数，得到矛盾。从而 n 的非递减的素数因子分解是唯一的。

3.7.1 一些有用的结果

- 引理2证明同余式两边除以同一整数的一个结果：
*同余关系对乘法保持同余，但对除法不一定。
- 例2：已知 $14 \equiv 8 \pmod{6}$, 但两边同时除以2，左边是 $\frac{14}{2} = 7$, 右边是 $\frac{8}{2} = 4$, 但 $7 \not\equiv 4 \pmod{6}$ 。

3.7.1 一些有用的结果

- **定理2:** 设 m 是正整数, 并设 a, b, c 是整数, 如果 $ac \equiv bc \pmod{m}$ 且 $\gcd(c, m)=1$, 那么 $a \equiv b \pmod{m}$ 。
- **证明:** 因为 $ac \equiv bc \pmod{m}$, $m \mid (ac - bc) = c(a - b)$. 由引理1, 因为 $\gcd(c, m)=1$, 因此 $m \mid (a - b)$, 从而有 $a \equiv b \pmod{m}$.

3.7.2 线性同余式

- 同余式 $ax \equiv b \pmod{m}$, 其中 m 是正整数, a 和 b 是整数, x 是变量, 称为**线性同余式**(linear congruence).
- 如果存在整数 $\bar{a}$ 使得 $\bar{a}a \equiv 1 \pmod{m}$, 那么 $\bar{a}$ 称为 a 模 m 的**逆元**(inverse of a modulo m).
- **定理3**: 如果 a 和 m 是互素的整数且 $m > 1$. 那么 a 模 m 的逆元存在. 进一步, 这个逆元在模 m 意义下是唯一的。(即存在唯一的正整数 $\bar{a} < m$, 且 $\bar{a}a \equiv 1 \pmod{m}$ 且任何 a 模 m 的逆元都与 $\bar{a}$ 模 m 同余)。

3.7.2 线性同余式

- 证明：由定理1，因为 $\gcd(a, m)=1$ ，存在整数 s 和 t ，使得 $sa + tm = 1$
- 这意味着 $sa + tm \equiv 1 \pmod{m}$ 。但 $tm \equiv 0 \pmod{m}$ ，所以 $sa \equiv 1 \pmod{m}$
- 结果， s 是 a 模 m 的逆元，这个逆元模 m 是唯一的。证明如下：
- 假设还存在另一个逆元 r ，有 $ra \equiv 1 \pmod{m}$
- 由3.4节的练习，有 $(sa - ra) \equiv (1 - 1) = 0 \pmod{m}$ ，即 $m|(sa - ra) = a(s - r)$ 。又因为 $\gcd(a, m)=1$ ，由引理1， $m|(s - r)$ ，故 s 和 r 模 m 同余。

3.7.2 线性同余式

- 例3：求3模7的逆元。
- 解：因为 $\gcd(3, 7)=1$ ，由定理3知，3模7的逆元存在。由欧几里德算法， $7 = 2 \cdot 3 + 1$ ，从而有 $(-2) \cdot 3 + 1 \cdot 7 = 1$ ，故-2是3模7的逆元。与-2同余的数还有5, -9, 12等都是3模7的逆元。

3.7.2 线性同余式

- 例4：（求解模线性方程）以下模线性方程的解是什么？

$$3x \equiv 4 \pmod{7}$$

- 解：由例3知， -2 是3模7的逆元，线性方程两边乘 -2 ，得 $(-2) \cdot 3x \equiv (-2) \cdot 4 \pmod{7}$.
- 因为 $-6 \equiv 1 \pmod{7}$ 和 $-8 \equiv 6 \pmod{7}$ ，所以， x 的解是 $x \equiv -8 \equiv 6 \pmod{7}$.
- 我们要确定满足 $x \equiv 6 \pmod{7}$ 的每个 x 都是一个解。设 $x \equiv 6 \pmod{7}$ ，由3.4节定理5，有

$$3x \equiv 3 \cdot 6 = 18 \equiv 4 \pmod{7}$$

故所有满足 $x \equiv 6 \pmod{7}$ 的 x 都是方程的解。

3.7.3 中国剩余定理

- **定理4：（中国剩余定理）** 设 $m_1, m_2, \dots, m_n$ 是两两互素的正整数且 $a_1, a_2, \dots, a_n$ 是任意整数，那么同余方程组
- $x \equiv a_1 \pmod{m_1}$
- $x \equiv a_2 \pmod{m_2}$
- $\vdots$
- $x \equiv a_n \pmod{m_n}$
- 有模 $m = m_1 m_2 \cdots m_n$ 唯一的解(即存在解 x 满足 $0 \leq x < m$ 并且任何其它解都模 m 与 x 同余)。

3.7.3 中国剩余定理

- 证明：我们将证明解存在并且唯一。为了构造联立方程组的解，首先设 $M_k = \frac{m}{m_k}$, $k = 1, 2, \dots, n$. 因为 m_i 和 m_k 互素（当 $i \neq k$ 时）， $\gcd(m_k, M_k) = 1$ ，由定理3，我们知道存在 y_k 是 M_k 模 m_k 的逆元，满足： $M_k y_k \equiv 1 \pmod{m_k}$.
- 我们现在构造方程组的联立解：
- $$x = a_1 M_1 y_1 + a_2 M_2 y_2 + \dots + a_n M_n y_n$$
- 我们将证明 x 是联立解。首先注意到 $M_j \equiv 0 \pmod{m_k}$ 对 $j \neq k$ 成立， x 中所有项除了第 k 项外，模 m_k 余数为0. 又因为 $M_k y_k \equiv 1 \pmod{m_k}$ ，有
- $$x \equiv a_k M_k y_k \equiv a_k \pmod{m_k}, \quad k = 1, 2, \dots, n$$
- 因此， x 是上述联立方程的解。该解的唯一性证明留作练习。

3.7.3 中国剩余定理

- 例6：设有 $x \equiv 2 \pmod{3}$, $x \equiv 3 \pmod{5}$, $x \equiv 2 \pmod{7}$ ，求解 x .
- 解：设 $m = 3 \cdot 5 \cdot 7 = 105$, $M_1 = \frac{m}{3} = 35$, $M_2 = \frac{m}{5} = 21$, $M_3 = \frac{m}{7} = 15$. 我们得到2是 $M_1 = 35$ 模3的逆元，因为 $35 \equiv 2 \pmod{3}$ ；1是 $M_2 = 21$ 模5的逆元，因为 $21 \equiv 1 \pmod{5}$ ；1是 $M_3 = 15$ 模7的逆元，因为 $15 \equiv 1 \pmod{7}$ 。因此联立解 x 为：
- $$x = a_1 M_1 y_1 + a_2 M_2 y_2 + a_3 M_3 y_3$$
- $$= 2 \cdot 35 \cdot 2 + 3 \cdot 21 \cdot 1 + 2 \cdot 15 \cdot 1$$
- $$= 233 \equiv 23 \pmod{105}$$
- 因此，23是最小的正整数联立解。

3.7.4 费马小定理

- **定理5:** 如果 p 是素数且 a 是一个不能被 p 整除的整数, 那么 $a^{p-1} \equiv 1 \pmod{p}$,
- 进一步, 对每一个整数 a , 我们有
$$a^p \equiv a \pmod{p}.$$

作业11

- 1. 将以下十进制数转换为二进制数，再转换为八进制数和十六进制数。
 - (1) 231 ; (2) 321 。
- 2. 用欧几里德算法求以下最大公因子： $\gcd(1000, 5040)$ 。
- 3. 求解模线性方程： $13x \equiv 5 \pmod{9}$ 。
- 4. 用中国剩余定理求以下联立同余方程的解：
 - $x \equiv 2 \pmod{3}; x \equiv 1 \pmod{4}; x \equiv 3 \pmod{5}$ 。

The End



机器人与信息自动化研究所

Institute of Robotics & Automatic Information System



南開大學
Nankai University